A forty-one-variable candidate equation

Positive integer unknowns; exact degree 4; a 170-operation evaluator.

The only unknowns are the forty-one separately named positive-integer variables

$$J,F,\alpha,f,g,T,X,Y,\xi,\eta,\zeta,\upsilon,\gamma,\vartheta,\psi,\varphi,\epsilon,$$
$$\ell,W,s,x,m,n,z,D,V,\delta,$$
$$\kappa,\lambda,\tau,\chi,\theta,\nu,Q,$$
$$N,O,P,S,A,i,\Gamma\in\mathbb{Z}_{>0}.$$

All other symbols below are polynomial abbreviations:

$$b=\beta J,\quad q=1+b,\quad C=q-F-\alpha,\quad U=Y\kappa,$$
$$c=\eta+O,\quad a=\lambda+Y,$$
$$E=\lambda,\quad k=\eta+\zeta,$$
$$G=i,$$
$$H=3+4a,\quad \Delta=\delta,\quad L=P,$$
$$M=sb,$$
$$R=G(\kappa-1)+J(\mu+\omega q),$$
$$I=\theta,\quad Z=\psi z,$$
$$\mathcal{P}=z^2-(1+\tau),$$
$$\mathcal{D}=D^2-\chi(1+N),$$
$$\mathcal{Q}=\nu-Q[\nu-\vartheta]-1,$$
$$\mathcal{R}=I^2-Q,\quad \mathcal{E}=\vartheta-V^2,$$
$$\mathcal{B}=\psi-M,\quad \mathcal{C}=\varphi-U,\quad \mathcal{G}=\epsilon-R,$$
$$\mathcal{L}=\ell-EO,\quad \mathcal{W}=W-[C(X+Kq)-Fq],$$
$$\mathcal{X}=s-q\kappa,\quad \mathcal{Y}=x-Eq,$$
$$\mathcal{Z}=m-f^2,\quad \mathcal{N}=n-c^2,$$
$$\mathcal{I}=Z-[\psi(\ell+k+\xi-\Gamma)+Yb-Ws],$$
$$\mathcal{J}=D-[\varphi L+H(S+x+\epsilon+1-k)],$$
$$\mathcal{K}=V-[m(A-\epsilon)-c],\quad \mathcal{O}=\delta-H-a^2,$$
$$\mathfrak{J}=\kappa-q^2,\quad \mathcal{F}=\lambda-XY,$$
$$\mathcal{S}=\tau-\ell(\ell+k),$$
$$\mathcal{U}=\chi-\varphi^2,\quad \mathcal{V}=\theta-ng,$$
$$\mathfrak{A}=\nu-\upsilon^2,\quad \mathfrak{B}=Q-\Delta(m-1),$$
$$\mathfrak{C}=N-\delta n,\quad \mathfrak{D}=G-[\alpha+(q-F)b],$$
$$\mathfrak{E}=O-Yk,\quad \mathfrak{F}=\Gamma-kW,$$
$$\mathfrak{G}=P-X-ac,\quad \mathfrak{H}=S-\varphi\gamma,$$
$$\mathfrak{I}=A-Tc.$$

The single Diophantine equation is

$$\begin{aligned}&\mathcal{P}^2+\mathcal{D}^2+\mathcal{Q}^2+\mathcal{R}^2+\mathcal{E}^2+\mathcal{B}^2+\mathcal{C}^2+\mathcal{G}^2\\&+\mathcal{L}^2+\mathcal{W}^2+\mathcal{X}^2+\mathcal{Y}^2+\mathcal{Z}^2+\mathcal{N}^2\\&+\mathcal{I}^2+\mathcal{J}^2+\mathcal{K}^2+\mathcal{O}^2+\mathfrak{J}^2+\mathcal{F}^2\\&+\mathcal{S}^2+\mathcal{U}^2+\mathcal{V}^2+\mathfrak{A}^2+\mathfrak{B}^2\\&+\mathfrak{C}^2+\mathfrak{D}^2+\mathfrak{E}^2+\mathfrak{F}^2+\mathfrak{G}^2+\mathfrak{H}^2+\mathfrak{I}^2=0.\end{aligned}$$

The four fixed integers. Every index, set, sum, product and floor below defines a constant, not another unknown. Put

$$\rho=2^{47},\quad B=\rho^{2011},\quad \beta=B-1,$$
$$G_m(x)=\sum_{j=0}^{m-1}x^j,\quad \Lambda=\rho^{1005}.$$

$$\begin{aligned}\mu&=\beta-G_{951}(\rho)\\&\quad-\rho^{960}G_{26}(\rho)-\rho^{1004},\\\omega&=\beta+8G_{917}(\rho)-4+(2^{45}-4)\Lambda\\&\quad+\rho^{960}G_9(\rho)+\rho^{987}G_8(\rho)+\rho^{1004},\\K&=\rho^{18}+\rho^{1004}+\rho^{2020}\\&\quad+2\Lambda\Phi(1/\rho),\end{aligned}$$

where

$$\chi_i=1+2^{10}(1+w_i+2^8n_i$$
$$+2^{17}p_i+2^{25}s_i)\quad(0\le i\le 941),$$
$$\pi(0)=0,\quad \pi(i)=1+\sum_{j=1}^{941}\left\lfloor\frac{2^{44}-1+\chi_j-\chi_i}{2^{44}}\right\rfloor\quad(1\le i\le 941).$$

$$\begin{aligned}\Phi(x)=&\sum_{i=0}^{941}\chi_i x^{\pi(i)}\\&+2^{10}x^{969}(x^9+2^{17})G_8(2x)\\&+2^{18}x^{942}(x^{18}+2^{17})G_9(2x).\end{aligned}$$

The rational intermediate in K cancels exactly; all four results are positive integers.

The 942 indexed fixed tiles. Use the actions u_j,v_j,d_j decoded below. For $d=0,1$, let

$$h(q,b)=2q+b+2,\quad \mathcal{S}_d=\{u_j:d_j=d,\ 0\le j<430\},$$
$$r_{d,q}=205-199d+|\mathcal{S}_d\cap\{0,\dots,q-1\}|.$$

An interval ending at -1 is empty. The table defines t_i for $0\le i\le 941$; all index ranges are fixed:

index i	tile $t_i=(w_i,n_i,p_i,s_i)$
0	$(2,435,3,1)$
$j+1$ $(j=2q+b<430)$	$((1-d_j)r_{0,u_j},h(q,b),d_jr_{1,u_j},v_j)$
$433-2a+b$ $(a,b=0,1)$	$(0,432a+b,0,432a+b)$
$\substack{423+2r_{d,q}+b\\ q\in\mathcal{S}_d,\ d,b=0,1}$	$(dr_{d,q},b,(1-d)r_{d,q},h(q,b))$
$923+2a+b$ $(a,b=0,1)$	$(0,432a+b,0,438)$
927	$(2,434,2,0)$
928	$(3,436,4,4)$
929	$(4,436,4,0)$
$496+t$ $(t=434,435,436)$	$(1,438,1,t)$
$933+3a+b$ $(a,b=0,1,2)$	$(a^2,437,b,437)$

A closed decoder for the 430 actions. Put $D_b(A,k)=\left\lfloor A/b^k\right\rfloor \bmod b$, where $a \bmod b = a-b\lfloor a/b\rfloor$. For $0\le q<215$, $b=0,1$, $j=2q+b$:

$$\mathcal{N}=\mathcal{A}\bmod 2^{636},\quad \mathcal{T}=\left\lfloor\mathcal{A}/2^{636}\right\rfloor,$$
$$\nu_j=D_2(\mathcal{N},j),\quad \kappa_j=(\sigma_j+2)^{1-\nu_j}\quad(0\le j<430),$$
$$\sigma_j=\sum_{k=0}^{j-1}\nu_k,\quad \Pi_j=\prod_{k=0}^{j-1}\kappa_k\quad(0\le j\le 430),$$
$$m_q=D_2(\mathcal{N},430+q),\quad \tau_q=\sum_{k=0}^{q-1}m_k,$$
$$F_q=14+m_q(D_{16}(\lfloor\mathcal{T}/\Pi_{430}\rfloor,\tau_q)-14),$$
$$u_j=(\sigma_{j+1}-(\lfloor\mathcal{T}/\Pi_j\rfloor \bmod \kappa_j)) \bmod 216,$$
$$v_j=D_2(F_q,2b),\quad d_j=D_2(F_q,2b+1).$$

Empty sums/products are 0/1. Here $(v_0,d_0,u_0)=(1,1,1)$; the constructor tile uses this actual first target, including its state renaming.

The integer $\mathcal{A}$ in base 10. Its complete decimal expansion is the concatenation of these lines:

```
12889260410359561746736540743515845134244260640012702033212621
95398074180784058571518488282119861023093355389406939414042892
97455773992193784795705001618777025966369137954859764532832722
75411843410809038713502633204617333246733616940964578272360901
56186216508504953649192752154519268024509952082950238865468307
22999632482769253980274091217460707829023970817244790340704449
09085581271192071656170732004937337082171141082486384269072884
72778087419663417942892520175630649792329265834490253035577533
96449689808375788457488971017043689875433162972390177936123032
89025657757489526342497438772045714099661532481123050235898973
27443609903800541199351405985754278307227777584454508422717998
33411356440933534607489268032843 7
```

The fixed integer $\mathcal{A}$ above is written in base 10. This equation is a candidate: the quotient witness construction and native intuitionistic PRA equivalence are not yet established. Its exact degree is certified by integer-polynomial calculations; the 170-operation count is for the unshifted circuit. Neither count is a minimality claim.

Exact 170-operation circuit

Each numbered line defines one gate over the integers. The inputs are the forty-one unknowns and the four fixed integers above. The ASCII names `alpha, xi, eta, zeta, upsilon, gamma, vartheta, psi, varphi, epsilon, beta, mu, omega, ell, Delta, delta, kappa, lambda, tau, chi, theta, nu, Gamma` refer to their displayed counterparts; `g` is the unknown g. The last gate is zero exactly when the displayed equation holds. The circuit input `ellNamed` is ℓ; gates `c, a` are the displayed abbreviations; input `WNamed` is the displayed W, while gate `WComputed` is $C(X + Kq) - Fq$. The inputs `s, x, m, n, z, D, V, delta` are the displayed $s, x, m, n, z, D, V, \delta$; `kappa, lambda, tau, chi, theta, nu, Q` are $\kappa, \lambda, \tau, \chi, \theta, \nu, Q$; `N, sumY, P, S, A, i, Gamma` are N, O, P, S, A, i, Γ. Gate `zScaled` is the abbreviation $Z = \psi z$. Gates `E, L, I` compute $XY, X + ac, ng$; the display uses $E = \lambda$, $L = P$, $I = \theta$. Intermediate gate names below are local abbreviations, not additional unknowns.

```
1.  b = J * beta
2.  q = 1 + b
3.  q2 = q * q
4.  q3 = q * kappa
5.  U = Y * kappa
6.  qF = q - F
7.  C = qF - alpha
8.  E = X * Y
9.  k = eta + zeta
10.  kY = Y * k
11.  c = eta + sumY
12.  c2 = c * c
13.  a = lambda + Y
14.  g15 = 4 * a
15.  H = 3 + g15
16.  g17 = a * a
17.  DeltaComputed = H + g17
18.  g19 = a * c
19.  L = X + g19
20.  ell = lambda * sumY
21.  M = s * b
22.  g25 = K * q
23.  g26 = X + g25
24.  g27 = C * g26
25.  g28 = F * q
26.  WComputed = g27 - g28
27.  g30 = ellNamed + k
28.  g31 = g30 + xi
29.  root_skewComputed = k * WNamed
30.  root_inner = g31 - Gamma
31.  root_scaled = psi * root_inner
32.  g33 = Y * b
33.  root_btq3 = WNamed * s
34.  root_sum = root_scaled + g33
35.  ZComputed = root_sum - root_btq3
36.  g38 = z * z
37.  g40 = ellNamed * g30
38.  g41 = 1 + tau
39.  A0 = g38 - g41
40.  g44 = qF * b
41.  GComputed = alpha + g44
42.  g46 = kappa - 1
43.  g47 = i * g46
44.  g48 = omega * q
45.  g49 = mu + g48
46.  g50 = J * g49
47.  R = g47 + g50
48.  g52 = k - epsilon
49.  r0 = g52 - 1
50.  g54 = varphi * gamma
51.  g55 = lambda * q
52.  g56 = S + x
53.  shift = g56 - r0
54.  g58 = varphi * P
55.  g59 = H * shift
56.  DstarComputed = g58 + g59
57.  g61 = D * D
58.  g62 = varphi * varphi
59.  g63 = delta * n
60.  g64 = 1 + N
61.  g65 = chi * g64
62.  A1 = g61 - g65
63.  f2 = f * f
64.  rho = m - 1
65.  QComputed = delta * rho
66.  I = n * g
67.  g71 = theta * theta
68.  g72 = g71 - Q
69.  g74 = T * c
70.  g75 = A - epsilon
71.  g76 = m * g75
72.  VComputed = g76 - c
73.  y2 = upsilon * upsilon
74.  g80 = V * V
75.  g81 = nu - vartheta
76.  g82 = Q * g81
77.  Fa = nu - g82
78.  auxResidual = Fa - 1
79.  firstSquare = A0 * A0
80.  mainSquare = A1 * A1
81.  auxSquare = auxResidual * auxResidual
82.  strongSquare = g72 * g72
83.  firstMain = firstSquare + mainSquare
84.  firstMainAux = firstMain + auxSquare
85.  sumSquares = firstMainAux + strongSquare
86.  difference = vartheta - g80
87.  constraint = difference * difference
88.  result = sumSquares + constraint
89.  scaleDiff = psi - M
90.  scaleCheck = scaleDiff * scaleDiff
91.  final = result + scaleCheck
92.  mainDiff = varphi - U
93.  mainCheck = mainDiff * mainDiff
94.  result16 = final + mainCheck
95.  rankDiff = epsilon - R
96.  rankCheck = rankDiff * rankDiff
97.  result17 = result16 + rankCheck
98.  ellDifference = ellNamed - ell
99.  ellConstraint = ellDifference * ellDifference
100.  wDifference = WNamed - WComputed
101.  wConstraint = wDifference * wDifference
102.  addEll = result17 + ellConstraint
103.  finalWithNamedValues = addEll + wConstraint
104.  cubeDifference = s - q3
105.  cubeConstraint = cubeDifference * cubeDifference
106.  productDifference = x - g55
107.  productConstraint = productDifference *
productDifference
108.  fSquareDifference = m - f2
109.  fSquareConstraint = fSquareDifference *
fSquareDifference
110.  cSquareDifference = n - c2
111.  cSquareConstraint = cSquareDifference *
cSquareDifference
112.  addCube = finalWithNamedValues + cubeConstraint
113.  addProduct = addCube + productConstraint
114.  addFSquare = addProduct + fSquareConstraint
115.  addCSquare = addFSquare + cSquareConstraint
116.  Z = psi * z
117.  zDifference = Z - ZComputed
118.  zConstraint = zDifference * zDifference
119.  dDifference = D - DstarComputed
120.  dConstraint = dDifference * dDifference
121.  vDifference = V - VComputed
122.  vConstraint = vDifference * vDifference
123.  deltaDifference = delta - DeltaComputed
124.  deltaConstraint = deltaDifference *
deltaDifference
125.  addZConstraint = addCSquare + zConstraint
126.  addDConstraint = addZConstraint + dConstraint
127.  addVConstraint = addDConstraint + vConstraint
128.  addDELTAConstraint = addVConstraint +
deltaConstraint
129.  qSquareDifference = kappa - q2
130.  qSquareConstraint = qSquareDifference *
qSquareDifference
131.  product2Difference = lambda - E
132.  product2Constraint = product2Difference *
product2Difference
133.  ellProductDifference = tau - g40
134.  ellProductConstraint = ellProductDifference *
ellProductDifference
135.  varphiSquareDifference = chi - g62
136.  varphiSquareConstraint = varphiSquareDifference *
varphiSquareDifference
137.  scaledIndexDifference = theta - I
138.  scaledIndexConstraint = scaledIndexDifference *
scaledIndexDifference
139.  ySquareDifference = nu - y2
140.  ySquareConstraint = ySquareDifference *
ySquareDifference
141.  qSignedDifference = Q - QComputed
142.  qSignedConstraint = qSignedDifference *
qSignedDifference
143.  add_qSquare = addDELTAConstraint +
qSquareConstraint
144.  add_product2 = add_qSquare + product2Constraint
145.  add_ellProduct = add_product2 +
ellProductConstraint
146.  add_varphiSquare = add_ellProduct +
varphiSquareConstraint
147.  add_scaledIndex = add_varphiSquare +
scaledIndexConstraint
148.  add_ySquare = add_scaledIndex + ySquareConstraint
149.  add_qSigned = add_ySquare + qSignedConstraint
150.  deltaProductDifference = N - g63
151.  deltaProductConstraint = deltaProductDifference *
deltaProductDifference
152.  rankSeedDifference = i - GComputed
153.  rankSeedConstraint = rankSeedDifference *
rankSeedDifference
154.  ySumDifference = sumY - kY
155.  ySumConstraint = ySumDifference * ySumDifference
156.  rootSkewDifference = Gamma - root_skewComputed
157.  rootSkewConstraint = rootSkewDifference *
rootSkewDifference
158.  offsetDifference = P - L
159.  offsetConstraint = offsetDifference *
offsetDifference
160.  gammaProductDifference = S - g54
161.  gammaProductConstraint = gammaProductDifference *
gammaProductDifference
162.  timeProductDifference = A - g74
163.  timeProductConstraint = timeProductDifference *
timeProductDifference
164.  add_deltaProduct = add_qSigned +
deltaProductConstraint
165.  add_rankSeed = add_deltaProduct +
rankSeedConstraint
166.  add_ySum = add_rankSeed + ySumConstraint
167.  add_rootSkew = add_ySum + rootSkewConstraint
168.  add_offset = add_rootSkew + offsetConstraint
169.  add_gammaProduct = add_offset +
gammaProductConstraint
170.  add_timeProduct = add_gammaProduct +
timeProductConstraint
```